Lists

Lists

- container
- mutable
- indexed
- can store different object types
  - strings
  - integers
  - floats
  - lists
  - etc.

Creating Lists

- list() function accepts a single iterable object argument such as a string, list, or tuple and returns a new list object
  - list('abc') -> ['a', 'b', 'c']
  - lst = ['a', 'b', 'c']

Indexes

- similar to that of a string
  - indexable
  - sliceable
- modifications can occur at a given index
  - lst[1] = 5

List Methods

- append(x)
  - adds an item to the end of a list
- extend([x])
  - adds all items to the end of the list
- insert(i, x)
  - insert x into list before position i
- remove(x) - removes first item from list with value x
- pop() - removes and returns last item in list
  - lst.pop()
  - val = lst.pop()
  - val = lst.pop(2)

List Methods

- sort()
  - sort the items of list in-place
- reverse() - reverse the elements of list in-place
- index(x) - returns index of first item in list with value x
- count - count the number of times value x is in list

List Methods

- max(list) - returns max element in list
- min(list) - returns minimum element in list
- sum(list) - get the sum of all elements in the list

List Methods

- any(list) - True if any element is True otherwise false
  - non-zero values will return True otherwise return False
- all(list) - True if element in the list is True or if dealing with numerical data non-zero. If the list is empty it will return False

Nesting Lists

- Lists can contain other lists

lst = 1, 2, 3], [4, 5, 6
lst[0][0] -> 1
lst[1][2] -> 6
lst[0] -> [1, 2, 3]

Nesting Lists

- How does this effect iteration?

List Comprehension

- new_lst = [expression for loop_variable_name in iterable]